CORE CURRICULUM & ARCHITECTURE REFERENCE

Python & Data Types Master Study Guide

A comprehensive deep dive into Python architecture, memory models, object references, type hierarchies, primitive & collection data structures, operations, and classroom mastery concepts.

⚙️ Python Execution Internals

Source to Bytecode pipeline, Lexical tokens, AST, Code Objects, .pyc caching, and the Python Virtual Machine (PVM).

🧠 CPython Memory & Reference Model

PyObject layout, reference counters, small integer caching, string interning, id() vs ==, and shallow vs deep copy.

📊 Complete Data Type Taxonomy

Numeric types (int, float, complex, bool), Sequences (str, list, tuple, range), Sets, Dictionaries, and NoneType.

🎯 Classroom 71 Questions & Traps

Answers to core conceptual questions, operator precedence, type casting edge cases, and algorithmic patterns.

Anil Yadav
B.Tech Computer Science & Engineering
Repository: `AnilYadav17/Python`

Module: 01_Basics & Mastery Series
Updated: October 2026
Format: Official Reference Guide

📖 Comprehensive Table of Contents

1. Introduction to Python & Features	Module 1	2. Compilation & Execution Pipeline (PVM)	Module 1
3. Variables & Object References	Module 2	4. Memory Management: id(), sys.getsizeof()	Module 2
5. Interning: Small Integers & Strings	Module 2	6. Master Classification Matrix of Data Types	Module 3
7. Numeric Types: int, float, complex, bool	Module 4	8. The NoneType Singleton	Module 4
9. Sequence Type: Strings (str)	Module 5	10. Sequence Type: Lists (list)	Module 6
11. Sequence Type: Tuples & NamedTuples	Module 7	12. Sequence Type: range Objects	Module 7
13. Set Types: set & frozenset	Module 8	14. Mapping Type: Dictionaries (dict)	Module 9
15. Type Casting: Implicit vs Explicit	Module 10	16. Operators & Evaluation Semantics	Module 11
17. Classroom 71 Questions & Answers	Module 12	18. Key Assignment Patterns & Cheat Sheet	Module 13

Curriculum Scope: This guide synthesizes classroom lectures (`01_Basics/Class_Work`), comprehensive topic assignments (`Assignments/01_Basics/`), core architectural mastery guides (`MASTER/01_Basics` , `MASTER/09_Python_Memory_Management`), the 30-day revision track, and the 71 core assessment questions.

1. Introduction to Python & Language Architecture

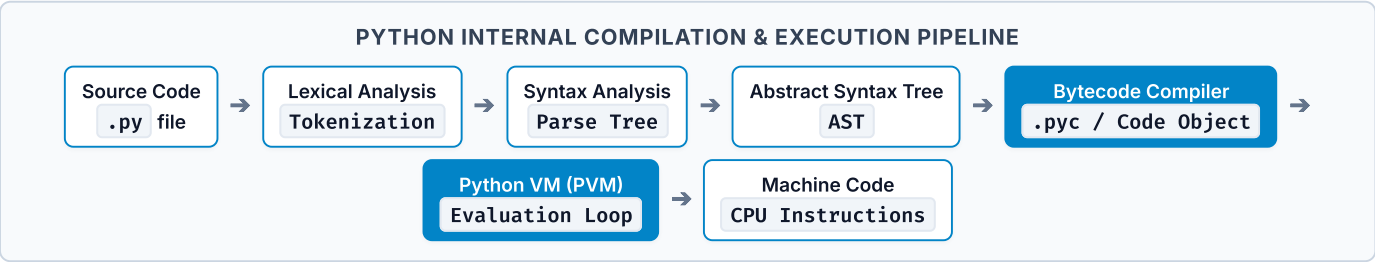
Python is a high-level, interpreted, dynamically typed, multi-paradigm programming language created by **Guido van Rossum** at CWI (Centrum Wiskunde & Informatica) in the Netherlands and released in **1991**. Named after the BBC comedy series *Monty Python's Flying Circus*, Python emphasizes code readability, developer ergonomics, and minimal syntactic ceremony.

Core Architectural Features

- **High-Level Abstraction:** Shields programmers from low-level hardware concerns such as manual memory allocation, register tracking, pointer arithmetic, and stack management.
- **Dynamically Typed:** Types are verified and bound at runtime. A variable is an un-typed identifier pointing to a strongly typed object in memory.
- **Strongly Typed:** Python will never implicitly convert types across incompatible boundaries (e.g., `"5" + 2` immediately raises a `TypeError`, unlike JavaScript's loose coercion).
- **Interpreted + Compiled Hybrid:** Source code is compiled to intermediate bytecode instructions before being interpreted by the Python Virtual Machine (PVM).
- **Automatic Memory Management:** Built-in reference counting backed by a 3-generation cyclic garbage collector automates memory allocation and deallocation.
- **Multi-Paradigm:** Fully supports Procedural, Object-Oriented (OOP), and Functional programming paradigms.
- **Platform Independence:** Code compiles to platform-independent bytecode. Any platform running a compatible PVM can execute the bytecode.

The Execution Pipeline: Source Code to CPU Execution

A common misconception is that Python is "purely interpreted." In reality, CPython follows a multi-stage compilation and execution pipeline:



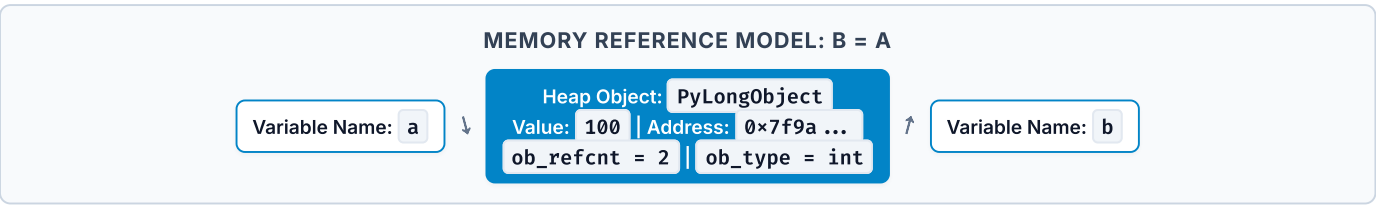
1. **Lexical Analysis (Tokenization):** The source code string is scanned character-by-character and broken down into basic syntactic units called *tokens* (keywords, identifiers, literals, operators, delimiters).
2. **Syntax Analysis (Parsing):** The parser verifies tokens against Python grammar rules, constructing a concrete *Parse Tree* (CST).
3. **Abstract Syntax Tree (AST):** The concrete syntax tree is transformed into a streamlined, high-level abstract syntax tree representing semantic logic.
4. **Bytecode Compilation:** The compiler converts AST nodes into platform-independent intermediate virtual machine instructions (bytecode). Bytecode instructions are packaged into **Code Objects** (containing `co_code`, `co_consts`, `co_varnames`) and cached on disk as `.pyc` files inside the `__pycache__` directory.
5. **PVM (Python Virtual Machine) Execution:** The PVM is a stack-based runtime engine that executes bytecode instructions sequentially by mapping them to host machine C-level instructions.

> 💡 **Why .pyc Files and __pycache__ Exist**
> When an imported module is compiled, CPython saves the compiled bytecode to `__pycache__/filename.cpython-XYZ.pyc`. On subsequent runs, CPython compares the timestamp of the source `.py` file with the `.pyc` file. If the source hasn't changed, Python skips tokenization, parsing, and compilation entirely, jumping straight to bytecode execution!

2. Variables, Identifiers & CPython Object References

Variables are Labels, NOT Storage Boxes

In languages like C or C++, a variable represents a fixed memory location whose bytes directly store a value. In Python, **variables are symbolic names (pointers or references)** bound to objects that reside in heap memory.



The Internal Anatomy of a CPython Object: `PyObject`

Every single entity in Python—whether an integer, float, string, function, or class—is an instance of the C struct `PyObject` (or `PyVarObject` for variable-length containers):

- `ob_refcnt` (Reference Count): An integer recording how many active references currently point to this object. When this hits zero, the object is immediately deallocated.
- `ob_type` (Type Descriptor Pointer): A pointer to the object's `PyTypeObject`, defining its type name, memory allocation size, and methods/dunders.
- `Payload / Data Buffer`: The actual value representation (e.g. `ob_digit[]` array for integers, `char[]` for strings).

Identifier Rules & Naming Conventions

- Must begin with an uppercase letter, lowercase letter, or underscore (`_`). Cannot start with a digit.
- Subsequent characters may consist of letters, digits, or underscores (`[a-zA-Z0-9_]`).
- Case-sensitive: `Total`, `total`, and `TOTAL` are three completely distinct identifiers.
- Cannot use reserved Python keywords (e.g., `for`, `while`, `if`, `def`, `class`, `is`, `in`, `None`).
- **PEP 8 Conventions:** `snake_case` for functions and variables, `PascalCase` for classes, `SCREAMING_SNAKE_CASE` for constants.

Memory Inspection: `id()` and `sys.getsizeof()`

As covered in classroom experiments (`Day4/size_and_id.py`), Python provides built-in tools to inspect memory addresses and allocation sizes:

```python
import sys

a = 10
b = 10.5
c = True
d = "Anil"
e = None

# Memory footprint in bytes on 64-bit CPython
print(sys.getsizeof(a))  # 28 bytes (PyLongObject overhead + digits)
print(sys.getsizeof(b))  # 24 bytes (PyFloatObject)
print(sys.getsizeof(c))  # 28 bytes (bool inherits from int)
print(sys.getsizeof(d))  # 53 bytes (PyUnicodeObject + Latin-1 characters)
print(sys.getsizeof(e))  # 16 bytes (NoneType singleton)

# Identity inspection (Virtual memory address of PyObject)
print(id(a))             # Memory address integer
```

CPython Caching & Optimization: Interning

🔢 Small Integer Caching [-5 to 256]

CPython pre-allocates and caches an array of integer objects for values between `-5` and `256` during interpreter startup. Any variable assigned a value in this range points to the identical cached singleton object in memory:

```
x = 256; y = 256
print(x is y)  # True (same id)

p = 257; q = 257
print(p is q)  # False (different ids)
```

🔤 String Interning

Strings that look like valid identifiers (alphanumeric + underscores) and compile-time string constants are automatically *interned* by CPython into a global dictionary. This allows comparison via fast pointer equality ($O(1)$) instead of char-by-char ($O(n)$):

```
s1 = "hello_world"
s2 = "hello_world"
print(s1 is s2)  # True

# Force manual interning
s3 = sys.intern("hello world!")
```

⚠️ The Golden Rule: `is` vs `==`

- `==` tests for **Value Equality** by calling `__eq__()`. It checks if two objects represent the same data.
- `is` tests for **Object Identity**. It checks if `id(a) == id(b)` (exact same memory pointer).
- **Never** use `is` to compare numbers or user strings! Always use `==`. Reserve `is` exclusively for singletons like `None`, `True`, and `False` (e.g. `if x is None:`).

3. Master Classification Matrix of Python Data Types

Every value in Python has an associated data type that dictates what operations can be performed, how memory is allocated, and whether the object's contents can be modified in-place.

Complete Data Type Taxonomy Table

Data Type	Category	Mutability	Ordering	Duplicates	Hashable	Syntax Literal Example	Primary Use Case
`int`	Numeric	IMMUTABLE	N/A	N/A	HASHABLE	`42`, `0b101`, `0xFF`	Exact discrete integer arithmetic
`float`	Numeric	IMMUTABLE	N/A	N/A	HASHABLE	`3.14159`, `2.5e-3`	Real numbers, scientific calculation
`complex`	Numeric	IMMUTABLE	N/A	N/A	HASHABLE	`3 + 4j`	Electrical engineering, Fourier transforms
`bool`	Numeric (int subclass)	IMMUTABLE	N/A	N/A	HASHABLE	`True`, `False`	Boolean flags, conditional branching
`NoneType`	Singleton	IMMUTABLE	N/A	N/A	HASHABLE	`None`	Absence of value, default return/argument
`str`	Sequence	IMMUTABLE	ORDERED	Allowed	HASHABLE	`"Anil"`, `'Python'`	Text processing, Unicode manipulation
`list`	Sequence	MUTABLE	ORDERED	Allowed	No	`[1, 2, "three", 4.0]`	Dynamic array, general-purpose collections
`tuple`	Sequence	IMMUTABLE	ORDERED	Allowed	HASHABLE*	`(10, 20)`, `(42,)`	Fixed records, dictionary keys, multi-returns
`range`	Sequence	IMMUTABLE	ORDERED	No (arithmetic)	HASHABLE	`range(0, 10, 2)`	Memory-efficient integer loop counters
`set`	Set Collection	MUTABLE	UNORDERED	Unique only	No	`{1, 2, 3, 4}`	Deduplication, Venn logic, $O(1)$ membership
`frozenset`	Set Collection	IMMUTABLE	UNORDERED	Unique only	HASHABLE	`frozenset([1, 2, 3])`	Immutable sets, set elements, dict keys
`dict`	Mapping	MUTABLE	ORDERED (3.7+)	Unique keys	No	`{"id": 1, "name": "A"}`	Key-value associative lookup ($O(1)$)

* Note on Tuple Hashability: A tuple is hashable if and only if all of its contained elements are also hashable. A tuple containing a list (e.g. `([1, 2], 3)`) is NOT hashable.

Mutability vs Immutability: Deep Technical Meaning

🔒 Immutable Objects

Once allocated in heap memory, the internal data bytes of an immutable object cannot be altered. Any operation that appears to modify an immutable object actually creates and returns a **brand new object** with a new memory address (`id`):

```
s = "Anil"
print(id(s))      # e.g., 140239120
s += " Yadav"
print(id(s))      # NEW address! Old object discarded
```

🔓 Mutable Objects

The internal state and contents of a mutable object can be modified in-place without altering the object's identity or memory address. Any other variable referencing the same object will observe the change immediately:

```
lst = [1, 2, 3]
print(id(lst))    # e.g., 140238992
lst.append(4)
print(id(lst))    # SAME address! Mutated in-place
```

4. Primitive & Scalar Data Types in Detail

1. Integer (`int`)

- **Arbitrary Precision:** In Python 3, integers have unbounded precision, constrained only by host RAM. Unlike C/Java where integers overflow at 32-bit ($2^{31}-1$) or 64-bit ($2^{63}-1$), Python handles calculations of arbitrary magnitude (e.g., 2^{1000}) seamlessly without overflow bugs.
- **Internal Representation:** Implemented in CPython as `PyLongObject`, using a variable-length array of 30-bit digits (`ob_digit[]`). CPython allocates additional digit slots dynamically as the number expands.
- **Radix Literals & Conversions:**
  - Binary: Prefix `0b` or `0B` (e.g., `0b1010` = 10). Conversion: `bin(10)`.
  - Octal: Prefix `0o` or `0O` (e.g., `0o12` = 10). Conversion: `oct(10)`.
  - Hexadecimal: Prefix `0x` or `0X` (e.g., `0xA` = 10). Conversion: `hex(10)`.
- **Visual Separators:** Underscores can be embedded for numeric readability: `population = 1_400_000_000`.

2. Floating-Point (`float`)

- **Standard Representation:** Conforms strictly to IEEE 754 64-bit double precision (1 sign bit, 11 exponent bits, 53 mantissa bits). Offers roughly 15 to 17 significant decimal digits of precision.
- **The Floating-Point Conundrum:**

```
print(0.1 + 0.2 == 0.3)       # False!
print(0.1 + 0.2)              # 0.30000000000000004
```

  Because fractions with denominators that are powers of 10 cannot always be expressed as finite base-2 binary fractions, floating-point arithmetic incurs slight rounding differences.
- **Solution for Financial / Scientific Equality:**

```
import math
math.isclose(0.1 + 0.2, 0.3)  # True
# Or use the decimal module for exact financial arithmetic
from decimal import Decimal
Decimal('0.1') + Decimal('0.2') == Decimal('0.3') # True
```

- **Special Float Literals:** `float('inf')`, `float('-inf')`, and `float('nan')` (Not a Number). Note that `float('nan') == float('nan')` is always `False`!

3. Complex Numbers (`complex`)

Python has native built-in support for complex numbers without external libraries. Written with real and imaginary components ending with `j` or `J`:

```
z = 3 + 4j
print(z.real)         # 3.0 (stored as float)
print(z.imag)         # 4.0 (stored as float)
print(z.conjugate())  # (3 - 4j)
print(abs(z))         # 5.0 (Calculates magnitude: sqrt(3^2 + 4^2))
```

> ⚠️ **Type Constraint Trap**
> Attempting to convert a complex number to `int` or `float` via `int(3+4j)` raises a `TypeError: can't convert complex to int`.

4. Boolean (`bool`) & Truth Value Testing

- `bool` is an explicit subclass of `int` in Python: `issubclass(bool, int) == True`.
- `True` corresponds numerically to `1`, and `False` corresponds to `0`.

- Therefore, boolean arithmetic evaluates as integer math:

```
print(True + True)  # 2
print(True * 5)     # 5
print(False - 1)    # -1
```

- **Falsy vs Truthy:** In Python, any object can be tested for truth value inside an `if` or `while` condition.

Falsy Values (Evaluate to False)	Truthy Values (Evaluate to True)
`None` , `False` Numeric zeros: `0` , `0.0` , `0j` Empty sequences: `""` , `[]` , `()` , `range(0)` Empty mappings & sets: `{}` , `set()`	Any non-zero number: `1` , `-5` , `0.001` Any non-empty string: `"hello"` , `" "` (space is truthy!), `"False"` Any non-empty collection: `[0]` , `(False,)` , `{"k": 0}`

5. The `NoneType` Singleton

`None` is the sole instance of the `NoneType` class. It represents the deliberate absence of a value, uninitialized state, or the default return value of functions that finish without an explicit `return` statement.

🎯 The Mutable Default Argument Trap

In Python, default parameter expressions in functions are evaluated **only once** when the function definition is executed, NOT every time the function is called!

```
# BAD PATTERN: Dangerous shared state
def add_item(item, target=[]):
    target.append(item)
    return target

# IDIOMATIC PATTERN: Using None as sentinel
def add_item_safe(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

5. Sequence Data Types: Strings (`str`)

A **string** (`str`) in Python is an ordered, immutable sequence of Unicode code points. Supported by **PEP 393** (Flexible String Representation), CPython stores strings using 1 byte (Latin-1), 2 bytes (UCS-2), or 4 bytes (UCS-4) per character depending on the largest code point, eliminating ASCII memory waste while supporting emojis and all world languages.

String Indexing & Slicing Mechanics

Strings support both positive (0 to $N-1$) and negative (-1 down to $-N$) indexing:

```
s = "PYTHON"
# Indices:   P (0, -6) | Y (1, -5) | T (2, -4) | H (3, -3) | O (4, -2) | N (5, -1)

# Slicing syntax: s[start:stop:step] (stop is EXCLUSIVE)
print(s[1:4])     # 'YTH'
print(s[::2])      # 'PTO' (every 2nd char)
print(s[::-1])     # 'NOHTYP' (reversal idiom)
print(s[-3:])      # 'HON' (last 3 chars)
```

Essential Built-In String Methods Catalog

Category	Methods	Description & Behavior
Case Conversion	`.upper()`, `.lower()`, `.title()`, `.capitalize()`, `.swapcase()`	Transforms character case. Returns a brand new string (original remains unaltered due to immutability).
Validation	`.isalpha()`, `.isdigit()`, `.isalnum()`, `.isspace()`, `.startswith(pfx)`, `.endswith(sfx)`	Returns boolean testing character composition or prefix/suffix matches.
Search & Count	`.find(sub)`, `.rfind(sub)`, `.index(sub)`, `.count(sub)`	`.find()` returns starting index or `-1` if missing. `.index()` raises `ValueError` if missing.
Trimming & Cleaning	`.strip()`, `.lstrip()`, `.rstrip()`	Removes leading and/or trailing whitespace (or specific characters if argument passed).
Tokenizing & Joining	`.split(sep)`, `.rsplit()`, `.splitlines()`, `'sep'.join(iter)`	`split()` returns list of substrings. `join()` concatenates an iterable with separator ($O(n)$ time).
Substitution	`.replace(old, new[, count])`	Substitutes occurrences of substring. Returns new string.

String Formatting Mechanisms

Modern: f-strings (Python 3.6+)

Fastest and cleanest formatting syntax evaluated directly at runtime via specialized bytecode:

```
name = "Anil"; score = 98.456
# Precision formatting
print(f"{name}: {score:.2f}")   # Anil: 98.46
# Alignment and padding
print(f"{name:>10}")            # '      Anil'
# Debug expression (3.8+)
print(f"{score=}")              # score=98.456
```

Legacy: % and .format()

Historical formatters retained for backwards compatibility across older Python codebases:

```
# %-formatting (printf style)
print("%s scored %d" % ("Anil", 98))

# str.format()
print("{} has {:.1f}".format("Anil", 98.456))
```

Practical Classroom Algorithms on Strings (from Assignments)

🔒 1. Bank Account / Credit Card Number Masking (Assignment 26)

```
card = "9876543210987654"
masked = "*" * (len(card) - 4) + card[-4:]
print(masked)  # ************7654
```

🔠 2. Manual Title Case without using `.title()` (Assignment 25)

```
sentence = "python programming from scratch"
words = sentence.split()
capitalized_words = [w[0].upper() + w[1:].lower() for w in words]
result = " ".join(capitalized_words)
print(result)  # 'Python Programming From Scratch'
```

📦 3. Run-Length Decompression / String Expansion (Assignment 30)

```
encoded = "a3b2c4"  # Output desired: 'aaabbcccc'
expanded = ""
for i in range(0, len(encoded), 2):
    char = encoded[i]
    count = int(encoded[i+1])
    expanded += char * count
print(expanded)  # 'aaabbcccc'
```

6. Sequence Data Types: Lists (`list`)

A **list** (`list`) is an ordered, mutable sequence of heterogeneous Python objects. It is the most versatile general-purpose container in Python.

Internal Memory Architecture: Dynamic Array of Pointers

In CPython, a list is implemented as a `PyListObject`, which wraps a dynamically resized array of object pointers (`PyObject** ob_item`).

- **Pointer Array:** The list itself does not store the raw data values contiguously; it stores a contiguous array of 8-byte memory addresses (pointers) pointing to individual objects located elsewhere in heap memory.
- **Over-Allocation Growth Pattern:** To guarantee amortized $O(1)$ append times, CPython allocates more memory slots than immediately requested. When capacity is exceeded, it expands using the formula: `new_allocated = (newsize >> 3) + (newsize < 9 ? 3 : 6) + newsize`.
- **Time Complexities:**
  - Random index access (`lst[i]`): $O(1)$ constant time.
  - Append to end (`lst.append(x)`): $O(1)$ amortized.
  - Pop from end (`lst.pop()`): $O(1)$.
  - Insert/Delete at index 0 (`lst.insert(0, x)` or `lst.pop(0)`): $O(n)$ linear time because all subsequent pointers must shift.

Comprehensive List Methods Catalog

Operation / Method	Time Complexity	Description & Key Behavior
`.append(item)`	$O(1)$ amortized	Appends a single element to the end of the list in-place.
`.extend(iterable)`	$O(k)$ where k=len	Iterates over the given iterable and appends each element individually.
`.insert(index, item)`	$O(n)$	Inserts element at the designated index, shifting remaining elements right.
`.remove(val)`	$O(n)$	Searches and removes the *first* occurrence of value. Raises `ValueError` if missing.
`.pop([index])`	$O(1)$ end / $O(n)$ mid	Removes and returns item at index (default is last item: `index=-1`).
`.clear()`	$O(n)$	Removes all items from the list, resetting size to 0.
`.index(val)`	$O(n)$	Returns first index of value. Raises `ValueError` if not present.
`.count(val)`	$O(n)$	Returns total number of occurrences of value in list.
`.sort(key=None, reverse=False)`	$O(n \log n)$	Sorts list **in-place** using Timsort algorithm. Returns `None`!
`.reverse()`	$O(n)$	Reverses elements of list **in-place**. Returns `None`.
`sorted(iterable)`	$O(n \log n)$	Built-in function: returns a **brand new sorted list**, leaves original untouched.

⚠️ The List Replication Trap: `[[0] * 3] * 3`

When you write `grid = [[0] * 3] * 3`, Python duplicates the *same inner list reference* three times! If you mutate one row, all rows change simultaneously:

```
grid = [[0] * 3] * 3
grid[0][0] = 99
print(grid)  # [[99, 0, 0], [99, 0, 0], [99, 0, 0]]  -- BUG!

# CORRECT PATTERN: Use List Comprehension
correct_grid = [[0 for _ in range(3)] for _ in range(3)]
correct_grid[0][0] = 99
print(correct_grid)  # [[99, 0, 0], [0, 0, 0], [0, 0, 0]]  -- SUCCESS!
```

Practical Algorithms on Lists (from Assignments & Exam Prep)

🏔️ 1. Peak Element Detection (Assignment 32)

```
arr = [10, 20, 15, 2, 23, 90, 67]
n = len(arr)
peaks = []
for i in range(n):
    if i == 0 and arr[0] >= arr[1]: peaks.append(arr[i])
    elif i == n - 1 and arr[-1] >= arr[-2]: peaks.append(arr[i])
    elif arr[i] >= arr[i-1] and arr[i] >= arr[i+1]: peaks.append(arr[i])
print(peaks)  # [20, 90]
```

👑 2. Leaders in an Array in $O(N)$ Time (Assignment 31)

```
arr = [16, 17, 4, 3, 5, 2]
leaders = []
max_right = float('-inf')
for x in reversed(arr):
    if x > max_right:
        leaders.append(x)
        max_right = x
leaders.reverse()
print(leaders)  # [17, 5, 2]
```

7. Sequence Data Types: Tuples & NamedTuples

Tuples (`tuple`)

A **tuple** is an ordered, immutable sequence of heterogeneous objects. Tuples are constructed using commas (parentheses are optional for grouping): `t = (1, 2, 3)` or `t = 1, 2, 3`.

Core Properties & Gotchas

- **Single-Element Tuple Trait:** A single element inside parentheses is parsed as an expression, not a tuple! A trailing comma is **mandatory**:

```
x = (42)    # Evaluates to int!
y = (42,)   # Tuple of length 1
```

- **Immutability of Reference vs Mutability of Content:** A tuple's references cannot change, but if an element inside is mutable, that inner element can still be modified:

```
t = ([1, 2], 3)
t[0].append(99)     # Perfectly valid! t is now ([1, 2, 99], 3)
```

- **The Infamous `t[0] += [4]` Anomaly:**
  Executing `t = ([1, 2], 3); t[0] += [4]` executes the in-place extension on the inner list (mutating it to `[1, 2, 4]`), but then attempts to re-assign the list reference back to `t[0]`. Because tuples reject item assignment, Python raises `TypeError: 'tuple' object does not support item assignment`—leaving the list mutated in memory despite the error!
- **Memory & Performance Advantage:** Because tuple sizes are fixed at creation, CPython allocates exactly the required memory without over-allocation buffers, and maintains a free list of recycled tuples up to length 20, making tuple allocation faster than lists.

Sequence Unpacking

```
# Basic unpacking
a, b, c = (10, 20, 30)

# Extended unpacking using starred expression (*)
first, *middle, last = [1, 2, 3, 4, 5]
print(first)   # 1
print(middle)  # [2, 3, 4] (always unpacked as a list)
print(last)    # 5
```

NamedTuples (`collections.namedtuple`)

As covered in **Assignment 37**, `namedtuple` combines the memory efficiency and immutability of tuples with clean attribute access:

```python
from collections import namedtuple

# Define record schema
Employee = namedtuple('Employee', ['id', 'name', 'dept', 'salary'])

emp1 = Employee(101, "Anil Yadav", "Engineering", 95000.0)
print(emp1.name)    # Attribute access: 'Anil Yadav'
print(emp1[3])      # Index access: 95000.0

# Practical query: Max salary employee in list
team = [
    Employee(1, "Alice", "Dev", 80000),
    Employee(2, "Bob", "QA", 72000),
    Employee(3, "Charlie", "Dev", 91000)
]
top_earner = max(team, key=lambda e: e.salary)
print(top_earner.name)  # Charlie
```

`range` Objects

The `range` type represents an immutable arithmetic progression of integers.

- **Constant Memory:** Regardless of whether it covers 10 numbers or 10 billion numbers, a `range` object always occupies the exact same fixed amount of memory (48 bytes on 64-bit systems) because it stores only `start`, `stop`, and `step` values.
- **$O(1)$ Operations:** In Python 3, index lookups (`r[5]`) and membership checks (`999 in r`) are calculated mathematically in $O(1)$ time without iterating!

8. Set Data Types: `set` & `frozenset`

A **set** is an unordered, mutable collection of unique, hashable objects. Mathematical set theory operations are implemented at C-speed using hash tables.

Hashability Requirement

To reside in a set, an object must be **hashable**: it must possess a hash code that never changes throughout its lifetime (implementing `__hash__()`) and be comparable to other objects (implementing `__eq__()`).

- Mutable objects (like lists, sets, and dictionaries) are unhashable: `{[1, 2]}` raises `TypeError: unhashable type: 'list'`.
- To create an empty set, you **must** write `s = set()`. Writing `s = {}` creates an empty dictionary!

Mathematical Set Operations

Operation	Operator	Method Equivalent	Mathematical Meaning
Union	`A | B`	`A.union(B)`	All unique elements present in A, B, or both.
Intersection	`A & B`	`A.intersection(B)`	Elements present simultaneously in both A and B.
Difference	`A - B`	`A.difference(B)`	Elements present in A that are NOT in B.
Symmetric Diff	`A ^ B`	`A.symmetric_difference(B)`	Elements in A or B, but NOT in both (exclusive-OR).
Subset	`A ⩽ B`	`A.issubset(B)`	Returns True if all elements of A are inside B.
Superset	`A ⩾ B`	`A.issuperset(B)`	Returns True if A contains all elements of B.
Disjoint	N/A	`A.isdisjoint(B)`	Returns True if A and B have zero elements in common.

> ⚠️ **remove() vs discard()**
> - `s.remove(x)` removes element `x`. If `x` is not present in the set, it raises a `KeyError`.
> - `s.discard(x)` safely removes element `x` if present, and does nothing without error if absent.

The `frozenset` Type

A `frozenset` is an immutable, hashable version of a set. Because it is immutable, its contents cannot be changed after instantiation.

```
# Frozenset can be used as a dictionary key or placed inside another set!
fs1 = frozenset([1, 2, 3])
fs2 = frozenset([3, 4, 5])

nested_set = {fs1, fs2}  # Valid! Standard set of sets is illegal
cache = {fs1: "Result A", fs2: "Result B"}  # Valid dictionary keys!
```

9. Mapping Data Type: Dictionaries (`dict`)

A **dictionary** (`dict`) is an associative mapping data structure storing key-value pairs. Keys must be unique and hashable; values can be arbitrary Python objects.

CPython Compact Dict Architecture (Python 3.6+)

Historically, Python dictionaries were sparse hash tables that wasted substantial memory. Since Python 3.6, CPython uses a revolutionary **compact dictionary design**:

- **Dense Entry Table:** Key references, hash values, and value pointers are packed sequentially into a dense array: `entries = [ [hash, key_ptr, val_ptr], ... ]`.
- **Small Hash Index Array:** A compact indices array maps hash table buckets to indices in the dense table.
- **Benefits:** Reduces memory footprint by 20% to 25% and automatically guarantees **insertion order preservation** as an official language specification feature.

Core Dictionary Methods Catalog

Method / Operation	Complexity	Description & Behavior
`d[key]`	$O(1)$	Access value by key. Raises `KeyError` if key is missing.
`d.get(key, default=None)`	$O(1)$	Safe lookup. Returns default value instead of raising `KeyError`.
`d.setdefault(key, default)`	$O(1)$	Returns value if key exists; otherwise inserts `key: default` and returns default.
`d.update(other)`	$O(k)$	Merges key-value pairs from another dictionary or iterable into `d` in-place.
`d.pop(key[, default])`	$O(1)$	Removes key and returns its value. Raises `KeyError` if missing and no default provided.
`d.popitem()`	$O(1)$	Removes and returns the last inserted `(key, value)` tuple (LIFO order).
`d.keys()`, `.values()`, `.items()`	$O(1)$	Returns dynamic **view objects** that reflect real-time changes to the dictionary without copying.

Classroom Dictionary Patterns (Assignments 39 & 40)

📊 Frequency Counter / Character Histogram

```
text = "engineering"
freq = {}
for char in text:
    freq[char] = freq.get(char, 0) + 1
print(freq)  # {'e': 3, 'n': 3, 'g': 2, 'i': 2, 'r': 1}
```

🏥 Nested Dictionary Processing: Hospital Records (Assignment 40)

```
patients = {
    101: {"name": "Anil", "age": 22, "diagnosis": "Fever", "bill": 1500},
    102: {"name": "Ravi", "age": 45, "diagnosis": "Cardiac", "bill": 45000}
}
# Calculate total billing
total_revenue = sum(p["bill"] for p in patients.values())
print(f"Total Revenue: {total_revenue}")  # 46500
```

⚠️ The Dictionary Mutation during Iteration Trap

Modifying a dictionary's keys while iterating over it raises: `RuntimeError: dictionary changed size during iteration`. Always iterate over a list copy of keys: `for k in list(d.keys()): del d[k]`.

10. Type Casting & Operators Evaluation

Implicit Type Casting (Type Promotion)

Python automatically converts one operand into a wider compatible data type during arithmetic operations to prevent precision loss:

```
a = 10      # int
b = 2.5     # float
c = a + b   # c is promoted automatically to float: 12.5
d = True + 5 # d is promoted to int: 6
```

Explicit Type Casting (Conversion Functions)

Programmers explicitly invoke constructor functions to convert between types:

- `int(x, [base])`: Truncates floats toward zero (`int(3.9) == 3`, `int(-3.9) == -3`). Parses numeric strings (`int("100")`).
- `float(x)`: Converts integer or numeric string to floating-point (`float("3.14") == 3.14`).
- `str(x)`: Converts any object into its readable string representation via `x.__str__()`.
- `bool(x)`: Converts value according to truthiness rules.

> ⚠️ **Common Type Casting Traps**
>
> - `int("10.5")` raises `ValueError: invalid literal for int() with base 10: '10.5'`! You must convert to float first: `int(float("10.5"))`.
> - `bool("False")` evaluates to `True`! Any non-empty string is truthy in Python. Only `bool("")` is `False`.
> - `float(10+5j)` raises `TypeError: can't convert complex to float`.

Operator Precedence & Associativity Table

Precedence Level	Operators	Description	Associativity
1 (Highest)	`()`, `[]`, `{}`	Parentheses, grouping, indexing, slicing	Left-to-Right
2	`**`	Exponentiation	**Right-to-Left** (e.g. `2**3**2 = 2**9 = 512`)
3	`+x`, `-x`, `~x`	Unary positive, negation, bitwise NOT	Right-to-Left
4	`*`, `/`, `//`, `%`	Multiplication, division, floor division, modulo	Left-to-Right
5	`+`, `-`	Addition, subtraction	Left-to-Right
6	`<<`, `>>`	Bitwise left and right shifts	Left-to-Right
7	`&`	Bitwise AND	Left-to-Right
8	`^`	Bitwise XOR	Left-to-Right
9	`|`	Bitwise OR	Left-to-Right
10	`==`, `!=`, `<`, `<=`, `>`, `>=`, `is`, `in`	Comparisons, identity, and membership	Left-to-Right (Chained)
11	`not`	Logical NOT	Right-to-Left
12	`and`	Logical AND (short-circuit)	Left-to-Right
13 (Lowest)	`or`	Logical OR (short-circuit)	Left-to-Right

Short-Circuit Logical Evaluation & Return Semantics

Unlike languages where `&&` and `||` strictly return boolean `true/false`, Python's `and` and `or` operators return the **exact operand value** that determined the outcome:

- `A and B`: If `A` is falsy, returns `A` immediately without evaluating `B`. Otherwise, returns `B`.
- `A or B`: If `A` is truthy, returns `A` immediately without evaluating `B`. Otherwise, returns `B`.
- **Classroom Puzzle:** `10 and 0 or 5`
  Step 1: Evaluate `10 and 0` ➔ 10 is truthy, so evaluate second operand: returns `0`.
  Step 2: Evaluate `0 or 5` ➔ 0 is falsy, so evaluate second operand: returns `5`!
- **Chained Comparisons:** `5 > 3 > 1` evaluates as `(5 > 3) and (3 > 1)` ➔ `True and True` ➔ `True`.

11. Classroom 71 Questions & Master Interview Q&A

Selected high-yield conceptual questions and solutions derived directly from the classroom curriculum (`01_Basics/Class_Work/python_71_qs.txt`):

Q1: Why is Python called an interpreted language if it compiles to bytecode?

A: Python is a hybrid. Source code is first compiled into intermediate bytecode (`.pyc`). It is called interpreted because this bytecode is not compiled into native machine code ahead of time; instead, it is interpreted line-by-line at runtime by the Python Virtual Machine (PVM).

Q2: What is the exact compilation pipeline in Python from source to CPU?

A: Source Code (`.py`) ➔ Lexical Analysis (Tokens) ➔ Syntax Parsing (Parse Tree) ➔ Abstract Syntax Tree (AST) ➔ Bytecode Generation (Code Object / `.pyc`) ➔ Execution via PVM runtime loop.

Q3: Why are strings immutable in Python?

A: Immutability guarantees: (1) **Security & Thread Safety**—instances can be safely shared across threads without locks; (2) **Hash Stability**—strings can safely serve as dictionary keys and set elements; (3) **Memory Optimization**—enables CPython to intern identical strings to a single shared memory address.

Q4: What is the difference between `/` and `//` operators?

A: `/` performs true division and always returns a `float` (e.g. `10 / 2 == 5.0`). `//` performs floor division, rounding down towards negative infinity (e.g. `10 // 3 == 3` , and `-10 // 3 == -4`).

Q5: Why is `^` not the exponentiation operator in Python?

A: In Python, `^` is the bitwise XOR (exclusive-OR) operator (e.g., `5 ^ 3 == 6`). Exponentiation is performed using double asterisks: `**` (e.g., `2 ** 3 == 8`).

Q6: What is the output of `print(True + True)` and why?

A: Outputs `2` . Because `bool` is a subclass of `int` , `True` has an integer value of `1` . The `+` operator invokes integer addition: $1 + 1 = 2$.

Q7: What is the difference between `input()` and `print()`? Why does `input()` always return a string?

A: `print()` outputs data to `sys.stdout` . `input()` reads raw bytes from `sys.stdin` until a newline. It always returns a `str` to maintain generality and prevent unsafe automatic code evaluation. Numerical inputs must be explicitly cast: `int(input())` .

Q8: How does Python swap two variables without a third variable?

A: `a, b = b, a` . Python evaluates the right-hand side first, packing `(b, a)` into an anonymous tuple in memory, then unpacks that tuple into variables `a` and `b` on the left.

Q9: What happens when executing `x = 10; y = 10` vs `x = 300; y = 300` in terms of `id()`?

A: For `10` , `id(x) == id(y)` is `True` because 10 falls within CPython's small integer cache pool (`[-5, 256]`). For `300` , in interactive mode, separate `PyLongObject` instances are allocated, resulting in distinct memory addresses (`id(x) ≠ id(y)`).

Q10: Why does `sys.getrefcount(obj)` return a value 1 higher than expected?

A: Because passing the object as an argument into `sys.getrefcount(obj)` creates a temporary reference on the function's execution stack frame during the call, incrementing the counter by 1.

12. Assignment Patterns & Quick Reference Cheatsheet

Summary of Key Assignment Modules Completed

- **01_Input_Output (15 Assignments):** Basic formatting, f-strings, `sep` and `end` parameters, multiline input processing, resume and menu printers.
- **02_Conditions_and_Branching (8 Assignments):** Nested decision structures, leap year algorithms, grade evaluation, boundary validation.
- **03_Calculations_and_Operators (10 Assignments):** Formula evaluations, billing calculation with slab tariffs, insurance eligibility, compound interest.
- **04_Loops_and_Numbers:** Armstrong numbers, palindromes, prime detection, factorial, Fibonacci sequences, `for-else` and `while-else` search patterns.
- **05_Pattern_Programming:** Pyramid patterns, diamond stars, inverted triangles, numerical matrix shapes.
- **06_Strings (Assignments 24–30):** Character frequency, privacy masking, special character cleaners, substring algorithms, email validators.
- **07_Lists_and_Arrays (Assignments 31–36):** Peak element detection, array leaders, 2D matrix rotations, second largest element, list comprehensions.
- **08_Tuples_and_NamedTuples (Assignments 36–37):** Immutable coordinate pairs, `namedtuple` salary analysis, group aggregation.
- **09_Sets (Assignment 38):** Student club memberships, Venn union/intersection, deduplication pipelines.
- **10_Dictionaries (Assignments 39–40):** Shopping cart aggregation, hospital patient records, dynamic billing queries.

Master Quick Reference Cheatsheet

Task	Recommended Python Idiom	Key Benefit
Reverse a sequence	`seq[::-1]` or `reversed(seq)`	Fast C-level slicing or lazy generator
Deduplicate list	`list(dict.fromkeys(lst))`	Deduplicates *while preserving insertion order*
Safe Dict Lookup	`d.get(key, default)`	Prevents uncaught `KeyError` exceptions
Frequency Counting	`collections.Counter(iter)` or `d[k] = d.get(k, 0) + 1`	$O(n)$ linear aggregation
Swap variables	`a, b = b, a`	Atomic tuple unpacking without temporary variable
Check Singleton	`if val is None:`	Guaranteed pointer identity check
List Comprehension	`[x for x in seq if cond]`	Emits fast `LIST_APPEND` bytecode
Float Comparison	`math.isclose(a, b)`	Safe floating-point tolerance check
String Concatenation	`"".join(list_of_strings)`	$O(n)$ single pre-calculated memory allocation
Deep Copy Object	`copy.deepcopy(obj)`	Recursively copies nested state and avoids reference mutation